

Lab 04 — Vragen

Vraag 1 [Diagnose]

Een ontwikkelaar start `podman build -t api:1.0 .` vanuit `~/projecten/api/`, en zijn Dockerfile bevat `COPY ../gemeenschappelijk/config.yml /app/`. De build faalt met `possible escaping context directory error`. Leg uit waarom, en zeg waarom noch `-f`, noch een absoluut pad, noch `sudo` daar iets aan zullen veranderen. Wat is de correcte oplossing?

Vraag 2 [Analyse]

Onder Docker duurt de build van een Angular-project 4 minuten, waarvan 3 min 20 s getoond als `transferring context`. De map bevat `node_modules/` (900 MB) en `.git/` (200 MB). Een collega stapt over naar Podman: de build duurt nog maar 40 seconden, en hij besluit dat `.dockerignore` overbodig geworden is. Leg uit wat er onder Docker gebeurde, wat er onder Podman veranderd is, en waarom hij ongelijk heeft — door het **tweede** risico te noemen, los van de traagheid.

Vraag 3 [Begrip]

Een Dockerfile bevat `EXPOSE 8080`. De ontwikkelaar start `podman run -d mijn-api:1.0` en stelt vast dat `curl http://localhost:8080` niet antwoordt. Hij besluit dat de image kapot is. Heeft hij gelijk? Leg de exacte rol van `EXPOSE` uit.

Vraag 4 [Analyse]

Vergelijk deze twee Dockerfiles. Wat doet elk precies, en welke is correct voor een API-image?

```
# A
FROM docker.io/library/eclipse-temurin:21-jre
COPY api.jar /app/api.jar
RUN java -jar /app/api.jar
```

```
# B
FROM docker.io/library/eclipse-temurin:21-jre
COPY api.jar /app/api.jar
CMD ["java", "-jar", "/app/api.jar"]
```

Vraag 5 [Analyse]

Hier zijn twee images. Zeg voor elk wat de commando's `podman run img` en `podman run img --debug` opleveren.

```
# A
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

```
# B
CMD ["java", "-jar", "/app/api.jar", "--spring.profiles.active=prod"]
```

Zeg dan welke van de twee nog `podman run img sh` toelaat om te debuggen, en hoe je je uit de slag trekt met de andere.

Vraag 6 [Diagnose]

Een team klaagt dat zijn heruitrollen altijd tien seconden langer duren dan verwacht, dat Podman telkens `resorting to SIGKILL` toont, en dat Spring Boot nooit zijn *shutdown hooks* uitvoert. De Dockerfile eindigt met:

```
CMD java -jar /app/api.jar
```

Stel de diagnose, corrigeer, en leg uit waarom die ene regel volstaat om het symptoom te veroorzaken — en waarom het team het **niet** waarneemt op zijn testimage op basis van Alpine.

Vraag 7 [Analyse]

Een Dockerfile heeft de variabele `JAVA_OPTS` nodig bij het opstarten:

```
ENV JAVA_OPTS="-Xmx512m"  
ENTRYPOINT ["java", "$JAVA_OPTS", "-jar", "/app/api.jar"]
```

De container faalt met `Unrecognized option: $JAVA_OPTS`. Leg uit, en geef dan **twee** mogelijke correcties met vermelding van wat elke kost.

Vraag 8 [Begrip]

Een ontwikkelaar geeft het databasewachtwoord mee bij de build: `podman build --build-arg DB_PASSWORD=Secr3t! -t api:1.0 .`, met de uitleg dat een `ARG` niet in de image blijft. Is hij veilig? Verantwoord, en geef het commando dat je antwoord bewijst.

Vraag 9 [Analyse]

Een Maven-Dockerfile is zo geschreven:

```
COPY . /app  
WORKDIR /app  
RUN mvn -q package -DskipTests
```

Elke build duurt 6 minuten, zelfs als één enkel `.java`-bestand veranderd is. Herschrijf de instructies in de juiste volgorde, leg het cachemechanisme uit dat jouw versie sneller maakt, en zeg welke build toch traag zal blijven.

Vraag 10 [Diagnose]

```
RUN apt-get update  
RUN apt-get install -y curl vim  
RUN rm -rf /var/lib/apt/lists/*
```

Noem **drie** verschillende gebreken van deze drie regels, en geef de correcte versie.

Vraag 11 [Analyse]

Na alleen de laatste `COPY` van zijn Dockerfile gewijzigd te hebben, ziet een ontwikkelaar dat de build `--> Using cache` toont op de eerste acht stappen en dan de laatste twee herbouwt. De dag erna voegt hij een `ENV`-variabele toe op de **derde** positie en de hele build begint van nul. Verklaar beide gedragingen met dezelfde regel.

Vraag 12 [Begrip]

`COPY` en `ADD` lijken gelijkwaardig. Geef twee gedragingen eigen aan `ADD`, leg uit waarom de officiële aanbeveling is om `COPY` te gebruiken, en noem het enige geval waarin `ADD` gerechtvaardigd blijft.

Vraag 13 [Analyse]

Een Dockerfile bevat `USER 1000:1000` meteen na de `FROM`, vóór de instructies `COPY` en `RUN apt-get install`. De build faalt met `Permission denied`. Leg uit, en zeg waar `USER` thuishoort in een goed geschreven Dockerfile. En dan: in rootless-modus toont `podman top` voor die container `USER 1000` en `HUSER 100999`. Waarom dat tweede getal, en is de instructie `USER` nog nuttig aangezien "root" toch al geen root is?

Vraag 14 [Analyse]

Je collega beweert: "Je moet het aantal lagen zoveel mogelijk beperken, dus alles in één enkele `RUN` zetten." Bespreek: in welke gevallen heeft hij gelijk, in welke gevallen schaadt die regel de buildtijd en het gewicht van de overdrachten?